DS & Algo
-00940224-

- **Student 1:** Suheil Khourieh – [redacted]
- **Student 2:** [redacted] – [redacted]

Question 1:

Definition: Simple graphs $G = (V, E), H = (U, F)$ are called **isomorphic** if there exists a bijection map $\theta: V \to U$ that where $\forall u, v \in V$ it holds:

$$(u, v) \in E \Leftrightarrow \big(\theta(u), \theta(v)\big) \in F$$

Let it be marked $G \cong H$.
Intuitively you can think about isomorphic graphs as the same graph, but the names are different.
Definition: The complementary graph of G is the graph $\bar{G} = (V, \bar{E})$ where $\bar{E} = \{(u, v) | u, v \in V, (u, v) \notin E\}$.

Part 1:

Prove/Disprove: If an undirected graph $G = (V, E)$ is not connected, then $\bar{G}$ is connected.

Solution:

The claim is correct!

Let's assume by contradiction that the claim is incorrect, in other words, let's assume that there exists an undirected graph $G = (V, E)$ where G is not connected and that the complementary of G, $\bar{G}$ is also not connected:
G is not connected, thus it is possible to partition the vertex set V into two sets:

$$\exists V_1, V_2 \subset V : V_1 \cap V_2 = \emptyset \wedge V_1 \cup V_2 \equiv V$$

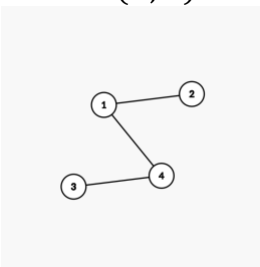
If we take a quick look to the subgraphs $G_1 = (V_1, E), G_2 = (V_2, E)$ we notice that both of them can be connected or not connected, it doesn't really matter.
$\bar{G}$ is actually $\overline{G_1}$ connected with $\overline{G_2}$ in the following way: $\forall v_1 \in V_1, v_2 \in V_2$ the edge $(v_1, v_2) \in \bar{E}$. $\Rightarrow$ We have proven for now that every two vertices, one from V_1 and the other from V_2, there exists an edge between them in the graph $\bar{G}$.
To complete the proof, we need to prove that for every two vertices from the same vertex set, there exists a path between them.
Assume that $|V_1| \geq 2$
Assume without loss of generality that $v_1, v_2 \in V_1 : v_1 \neq v_2$.
We need to show that there exists a path connect v_1 with v_2.
We have already proven that every pair of vertices, one from each vertex subset (V_1, V_2) are connected by an edge in $\bar{G}$.
Let $v_3 \in V_2$, we know that $e_1 = (v_1, v_3) \in \bar{E}$ and that $e_2 = (v_3, v_2) \in \bar{E}$.
Let's build a path by connecting the two edges e_1 and e_2 using v_3, the result is a path connecting v_1 with $v_2 \Rightarrow$
$every\ pair\ of\ vertices\ in\ V_1\ or\ V_2\ are\ connected\ using\ a\ vertex\ from\ the\ other\ subset.$
In case $|V_1| = |V_2| = 1$, we can easily see that the edge (v_1, v_2) will indeed be in $\bar{E}$ resulting in $\bar{G}$ to be connected. ■

Part 2:

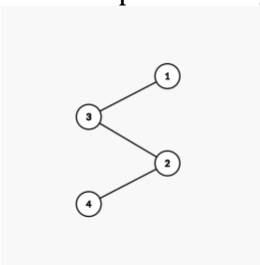
Give an example of an undirected graph $G = (V, E)$ with $n \geq 4$ vertices where $G \cong \bar{G}$

Solution:

Let $G = (V, E) : V = \{1,2,3,4\},\ E = \{(1,2), (3,4), (1,4)\}$:



The complementary graph of G, $\bar{G}$ will look like this:



Let's define the following bijection map $\theta: V \to E: \theta(v) = \begin{cases} 1, if\ v = 2 \\ 2, if\ v = 4 \\ 3, if\ v = 1 \\ 4, if\ v = 3 \end{cases}$

We can easily see that θ is indeed a bijection map.

Let's prove that the suggested map serve as a good example, to do so we need to show that $\forall u, v \in V, (u, v) \in E \Leftrightarrow \big(\theta(u), \theta(v)\big) \in \bar{E}$.

We know that $\bar{E} = \{(u, v) |\ u, v \in V, (u, v) \notin E\} = \{(1,3), (2,3), (2,4)\}$

Let's check the example we gave by going over all the edges in E and making the map we gave is correct:

$$(1,2) \in E \Rightarrow \big(\theta(1), \theta(2)\big) \overset{?}{\in} \bar{E} \Rightarrow (3,1) \in \bar{E}$$
$$(3,4) \in E \Rightarrow \big(\theta(3), \theta(4)\big) \overset{?}{\in} \bar{E} \Rightarrow (3,2) \in \bar{E}$$
$$(1,4) \in E \Rightarrow \big(\theta(1), \theta(4)\big) \overset{?}{\in} \bar{E} \Rightarrow (3,2) \in \bar{E}$$

$\Rightarrow \theta$ works as intended and the example we provided holds the requirements of isomorphic, according to it's definition.

Part 3:

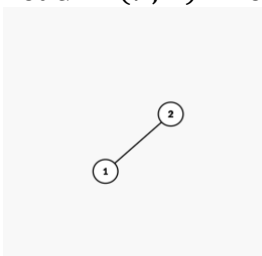
Prove/Disprove: If an undirected graph is connected, then $G \cong \bar{G}$.

Solution:

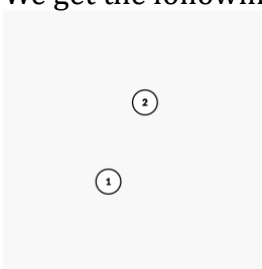
The claim is incorrect!

Disprove the claim using a counter example:

Let $G = (V, E)$ where $V = \{1,2\}, E = \{(1,2)\}$:



We get the following complementary graph $\bar{G}$:



Where $\bar{G} = (V, \bar{E}), \bar{E} = \emptyset$.

We need to prove that there doesn't exists **any** bijection map $\theta: V \to V$ that maintains the following for every $u, v \in V$: $(u, v) \in E \Leftrightarrow \big(\theta(u), \theta(v)\big) \in \bar{E}$.

Because θ must be a bijection map, there are only two options for it:

$$\theta_1(v) = \begin{cases} 2, if\ v = 1 \\ 1, if\ v = 2 \end{cases} \vee \theta_2(v) = \begin{cases} 1, if\ v = 1 \\ 2, if\ v = 2 \end{cases}$$

In both cases, θ_1 and θ_2, none of them maintain the definition of isomorphism simply because $|E| \neq |\bar{E}|$.

Part 4:

Prove/Disprove: If for an undirected graph $G = (V, E)$ it holds $G \cong \bar{G}$ then G is connected.

Solution:

The claim is correct!

Assume by contradiction that the claim is incorrect.

In other words, let's assume by contradiction that there exists an undirected graph $G = (V, E)$ where $G \cong \bar{G}$ and that G is not connected.

We know that $G \cong \bar{G} \Rightarrow$ Both G and $\bar{G}$ have the same structure, in other words either both are connected graphs or both are not connected graphs. According to our assumption, G is not connected. According to the claim proven in part 1 of this question, the complementary of G must be connected because G is not, resulting in having isomorphism between a connected graph and a disconnected graph which is not possible according to the definition of isomorphism$(\rightarrow\leftarrow)$. ■

Question 2:

Let $G = (V, E)$ be any graph. Recall that the distance between $u \in V$ to $v \in V$ is defined as the length of the shortest path between them and is denoted as $\delta(u, v)$. When there doesn't exist a path from u to v it is defined as $\delta(u, v) = \infty$. The diameter of graph is maximal distance in the graph and is denoted as D, in other words, $D = \max_{u,v \in V}\{\delta(u, v)\}$. In addition to that, denote the adjacency list of vertices $v \in V$ with $N(v)$, in other words, $N(v) = \{u \in V \mid (u, v) \in E\}$.

Part 1:

Let $G = (V, E)$ be any undirected connected graph and denote $|V| = n$. Find an upper tight bound as tight as possible for D depending on n.

Solution:

We know that D is the maximal distance in the graph, and the distance between any two vertices is the shortest path between them, the shortest path between any two vertices in the graph is a **simple path** (doesn't have any cycles) this can easily be proven:

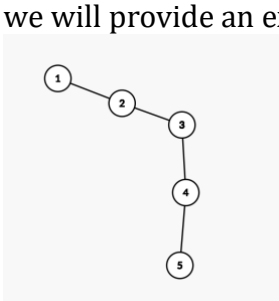
Claim: The distance between any two vertices $v, u \in V$ in an undirected graph $G = (V, E)$ is a path with no cycles.

Proof: Assume by contradiction that the claim is incorrect, assume that there exists a path p where p has a cycle starting from a vertex $c \in V$ and returning to it, $p =< u, \dots, some\ cycle\ from\ c\ to\ c, \dots, v >$, let p' be the path starting from u and getting to c in the same path as p, skipping the cycle starting from c and returning to c that in p and continuing to the next vertex, since $|c| \geq 1$ we get that $|p'| < |p|$, thus the shortest path between two vertices is a path with no cycles.

Back to our proof, we need to prove that $D < n$, to do so, let's assume by contradiction that there exists an undirected connected graph $G = (V, E)$, where the diameter of G is denoted as D is greater or equal to n, thus there exists a path p, connecting two vertices in V where p is the longest shortest path in the graph and the length of p is greater or equal to n, where n is the number of vertices.

Any path with any length m must include $m + 1$ vertices (not necessarily unique), thus the path p involves $n + 1$ vertices $\Rightarrow$ according to the pigeonhole principle, at least one vertex is involved at least twice resulting in creating a cycle in the path, meaning this is not a shortest path between the two vertices $(\rightarrow\leftarrow)$. ■

We have found that $D < n$, we need to prove that this is as tight as possible, to do so we will provide an example of a graph where $D = n - 1$:



We can easily see that $D = 4 = n - 1$ where $n = 5$.

$\Rightarrow D < n\ is\ indeed\ the\ tightest\ upper\ bound\ there\ is.$

Part 2:

Find an example of a graph family that its diameter is equal to the upper bound you found in the previous part.

Solution:

The graph family is the path graphs.
Path graphs are all the connected acyclic graphs where all the vertices are connecting in a single path and each vertex but the edges has a degree of exactly two where the edges have a degree of 1.
The example we provided in the previous part as part of the proof is part of the graph family we just described.

Part 3:

Prove/Disprove: Let $G = (V, E)$ be any undirected graph. If the diameter of G is greater than 2 then there exist $u, v \in V$, where $N(v) \cap N(u) = \emptyset \; and \; (u, v) \notin E$.

Solution:

The claim is correct!
Assume by contradiction that the claim is incorrect.
Assume that there exists an undirected graph $G = (V, E)$ where the diameter of G is greater than 2 and the following holds: $\forall u, v \in V, N(v) \cap N(u) \neq \emptyset \; \boldsymbol{or} \; (u, v) \in E$.
According to our assumption, there exists a path p connecting between two vertices let them be denoted as $u, v \in V$ where the length of the path is exactly the diameter of G and p is the shortest path connecting u and v, thus the length of p is greater than 2.
We will now prove that $N(v) \cap N(u) = \emptyset \; and \; (u, v) \notin E$.
Assume by contradiction that $N(v) \cap N(u) \neq \emptyset \Rightarrow \exists a \in V: (v, a) \in E \; and \; (u, a) \in E$, we can build the path $p' =< u, a, v >$ connecting u and v and resulting in a shorter path than p which can't be true.$(\rightarrow\leftarrow)$
$\Rightarrow N(v) \cap N(u) = \emptyset$.
Assume by contradiction that $(u, v) \in E \Rightarrow there \; exists \; a \; path \; p'$ connecting u and v where p' consists only of the single edge (u, v), resulting in a path shorter than p making p not the shortest path between u and v. $(\rightarrow\leftarrow)$
$\Rightarrow The \; claim \; is \; correct.$

Part 4:

Prove: Let $G = (V, E)$ be any undirected graph and denote $|V| = n$. If in G the degree of every vertex is at least $\left\lceil \frac{n-1}{2} \right\rceil$ then the diameter of G is at most 2.

Solution:

The claim is correct!
Assume by contradiction that the claim is incorrect.
Assume that there exists an undirected graph $G = (V, E): \forall v \in V, |N(v)| \geq \left\lceil \frac{n-1}{2} \right\rceil$ and that the diameter of G is at most 2.
The diameter of G is at most 2, thus $|V| \geq 3$ and according to the claim proven above $(Question \; 2 \; part \; 3), \exists v_1, v_2 \in V: N(v_1) \cap N(v_2) = \emptyset \wedge (v_1, v_2) \notin E$.
$\Rightarrow |V| \geq |N(v_1)| + |N(v_2)| + |v_1| + |v_2| \geq 2 \cdot \left\lceil \frac{n-1}{2} \right\rceil + 2 \geq n + 1 \Rightarrow |V| \geq n + 1$.
$\Rightarrow$ We got that $|V| \geq n + 1$ but it is given that $|V| = n$. $(\rightarrow\leftarrow)$
The claim is correct. ■

Question 3:

In all the sections below, an algorithm is described, and you should implement it using pseudocode. As you know, to implement a graph algorithm in the computer you need to represent the graph, and the graph representation may affect the time-complexity of the algorithm. In the tutorial we saw two graph representations (Adjacency matrix representation and adjacency list representation) and in each section in this question, you may choose the representation of the input graph. As you know, in those representations, in case there are n vertices, the vertices are numbered from 1 to n randomly.
Implement in pseudocode the following algorithms with as low of a time-complexity as possible. For each section, you should provide an explanation on how the algorithm works on the chosen representation and on the time-complexity.

Part 1:

Algorithm $Is_Path_k(G, P)$ that takes as an input any graph G in the chosen representation and an array P of size k that includes the names of vertices. The algorithm returns $True$ if the array P is a path in G, $False$ otherwise.

Solution:

Let G be any graph represented using the **adjacency matrix representation**.
This is the pseudocode of the algorithm:

```
Is_Path_k(G,P):
for i=0 to P.length()-1 do:
 if G.A[P[i]][P[i+1]]==0 do:
  return false
return true
```

Explanation:

Correctness:

The algorithm works by iterating over the path P and checking whether each pair of following vertices have an edge in the graph by checking the adjacency matrix of G. If when iterating a pair following vertices are found with no edge in the adjacency matrix, false is returned, otherwise true will be returned when the loop is done.

Graph-representation:

We choose to represent the graph using an adjacency matrix, simply because for each pair of vertices we can check whether an edge exists in the graph connecting them or not in $\Theta(1)$ time. Unlike the adjacency list which will force us to iterate over the list in order to check the existence of an edge ⇒ higher time-complexity.

Time-complexity:

The run-time complexity of the algorithm Is_Path_k is $O(k)$ where k is the size of the array P, simply because the algorithm iterates over the array ($k - 1$ $iterations$) and each iteration it checks an if statement that takes $O(1)$ as mentioned above, resulting in a total of $O(k)$.
Since k is a known constant, we get $O(1)$.

Part 2:

Algorithm $Num_Deg_d(G)$ that takes as an input any undirected graph G in the desired representation. The algorithm returns the number of vertices in G that have a degree of d. Be careful d is a constant that does not depend on the graph G.

Solution:

Let G be any undirected graph represented using the **adjacency list representation**. This is the pseudocode of the algorithm:

```
Num_Deg_d(G):
counter = 0
for v∈V do:
 if length(A[v]) == d do:
  counter++
return counter
```

Explanation:

Correctness:

The algorithm works by iterating over all the vertices in the graph G, for each vertex it checks it's degree using the adjacency array for the vertex, in other words, the length of the array is the degree of the vertex.

Graph-representation:

We choose to represent the graph using the adjacency list representation, we choose this form of representation because we can get the degree of any vertex in $O(1)$ time. On the other hand, the adjacency matrix representation does not have a direct $O(1)$ way to check the degree of any vertex, in the matrix representation we will need to iterate over the "row" of the vertex in the matrix and sum the values (since 1 indicates there is an edge and 0 otherwise) in order to get the degree of the vertex.

Time-complexity:

We can easily see that the time complexity of the algorithm is $O(n)$ where n is the number of vertices in the graph, this is true because we need to iterate over all the vertices in the graph and for each one we do $O(1)$ operations in order to get the degree of the vertex, exactly how we described above.

Part 3:

Algorithm $Complement(G)$ that takes any directed graph G in the desired representation. The algorithm returns G^c represented in the same way G is represented.

Solution:

Let G be any directed graph represented using the **adjacency matrix representation**. This is the pseudocode of the algorithm:

```
Complement(G):
A_complement = new Matrix of size VxV
for v in V do:
 for u in V do:
  if G.A[v][u] == 1 do:
   A_complement[v][u] = 0
  else:
   A_complement[v][u] = 1
return new graph Gᶜ =(V,A_complement)
```

Explanation:

Correctness:

The algorithm works by creating a new adjacency matrix and then iterating over all the vertices in the graph G, for each vertex it loops over all the vertices and checks if there exists an edge between the pair of vertices using the adjacency matrix of G, this check takes $O(1)$ time, if the edge exists the corresponding place in the new adjacency matrix is set to 0, if there is no edge, the corresponding place in the new adjacency matrix is set to 1.

When the nested loops are over, a new graph is created and returned using the same vertices set and the new adjacency matrix resulting in a new graph that is the complement to the original one and returned using the same representation G is using.

Graph-representation:

We choose to represent the graph using the adjacency matrix representation simply because we can iterate over **all the possible edges** (those that exist and those that don't) in $O(|V|^2)$.

Time-complexity:

The time-complexity of the algorithm is $O(|V|^2)$, building a new matrix is $O(1)$, iterating over all the vertices is $O(|V|)$, for each vertex we iterate over all the vertices again, thus $O(|V|)$, each time doing some $O(1)$ operations. ⇒ Resulting in a total of $O(|V|^2)$ exactly as expected.

Question 4:

Definition: Given an undirected and connected graph $G = (V, E)$, we say that $c \in V$ is the center of the graph if: $c \in \arg\min_{u \in V} \max_{v \in V} \delta(u, v)$.

In other words, c is the center of the graph if it minimizes the distance from the vertex that is furthest from it.

Let $T = (V, E)$ be a tree where $|V| \geq 3$. We will define $V' = \{v \in V : \deg_T(v) > 1\}$ in other words, V' is the set of vertices that their degree in T is greater than 1.

Let $T' = (V', E')$ be the sub-graph of T that is induced using V'.

Part 1:

Prove\Disprove: $c \in V$ is the center of T if and only if $c \in V'$ and c is the center of T'.

Solution:

The claim is correct!

Before proving the claim, let's prove the following lemmas:

Lemma 1: The set of vertices $l = V \backslash V'$ are only leaves or in special cases will include the root of T, the original tree.

Proof: This is true because in a tree, only leaves and in special cases the root to have a degree that is smaller or equal to one, making the set l include only leaves and the root sometimes. ■

Lemma 2: $\forall v_1 \in V, u = \arg\max_{v_2 \in V} \delta(v_1, v_2) \Rightarrow u \in V \backslash V'$

In other words, the vertex that maximizes the distance with any vertex in T must be a leaf or the root in special cases.

Proof: Assume by contradiction that u is not a leaf or the root when the root of the tree is of degree 1.

Because $u \notin V \backslash V'$ then $\exists v_2 \in V : v_2 \; is\ not\ in\ the\ path\ from\ v_1\ to\ u$, thus by creating the path $v_1\ to\ u\ and\ then\ to\ v_2$ we get a longer path than the one suggested making the vertex u not maximizing the distance of v_1 $(\rightarrow\leftarrow)$.

$\Rightarrow\ u\ must\ be\ a\ leaf\ or\ the\ root\ in\ case\ the\ root\ is\ of\ degre\ 1.$

Back to the question, we need to prove that:

(1) $c \in V\ is\ a\ center\ in\ T \Rightarrow c \in V' \wedge c\ is\ a\ center\ in\ T'$

(2) $c \in V'\ is\ a\ center\ in\ T' \Rightarrow c \in V\ is\ a\ center\ in\ T$

Proof of (1):

Assume by contradiction that there exists $T(V, E)$ a tree where $|V| \geq 3$ where $c \in V$ is a center in T and $c \notin V'$ or c is not a center in T'.

Proof that c is a center in $T \Rightarrow c \in V'$:

Assume by contradiction that c is the center in T yet it is not part of the tree $T'(\notin V')$.

If $c \in V \wedge c \notin V'$ then it is in the set defined in lemma 1 $l = V \backslash V'$, according to the lemma c must be a leaf or the root of the tree only if it's degree is 1.

Let u be the parent of c in case c is a leaf or the child of c in case it is the root.

Then u minimizes the maximum distances better than c making u a better candidate than c, making sure that c is not the center of T. $(\rightarrow\leftarrow)$

$c \notin l \Rightarrow c \in V'$ ■.

Proof that c is a center in $T \Rightarrow c$ *is a center in* T':
Assume by contradiction that c is a center in T and that c is not a center in T'.
Let's mark $d_T(v) = maximum\ distance\ v\ can\ have\ in\ the\ tree\ T, \forall v \in V$.
According to lemma 2, every vertex in any tree gets its maximum distance using a simple path from it to another leaf or the root in case the root is of degree 1.
Thus, $d_{T'}(v) = d_T(v) - 1, \forall v \in V'$.
If c provides the minimum of $d_T(v)$ in $T, \forall v \in V$, that is $d_T(c) \leq d_T(v), \forall v \in V$, then by removing 1 we get $d_T(c) - 1 \leq d_T(v) - 1, \forall v \in V$, but
$d_{T'}(c) = d_T(c) - 1\ and\ d_{T'}(v) = d_T(v) - 1, \forall v \in V$, thus $d_{T'}(c) \leq d_{T'}(v), \forall v \in V'$ making c a center of T'.

Proof of (2):
We know that c is a center in T', meaning that $d_{T'}(c) \leq d_{T'}(v), \forall v \in V'$.
According to lemma 2, any vertex in T' gets its maximum distance by a path to a leaf or the root in a special case described above, thus $\forall v \in V'$, $d_{T'}(v)$ is set using another vertex that is a leaf or the root, this leaf/root in T' can't be a leaf/root in T, because if it was, it will not be included in T' according to lemma 1.
Thus, $d_{T'}(v)$ will grow by one when comparing it in T, this is true for every vertex in $v \in V'$, resulting in $d_{T'}(c) \leq d_{T'}(v), \forall v \in V' \Rightarrow d_T(c) + 1 \leq d_T(v) + 1, \forall v \in V'$, we are left to handle vertices that are not part of V' and are part of V, this is the exact set of vertices described above, $l = V \backslash V'$, we have already proven that $v \in l$ cannot be a center in T, making it not even a close candidate to being a center of T. ■

$$\Rightarrow if\ c \in V'\ is\ a\ center\ of\ T' \Rightarrow c \in V\ is\ a\ center\ of\ T$$

⇒ We have proven both directions of the claim, making the claim indeed correct. ■

Part 2:
Prove\Disprove: The total number of centers in a tree is 1 or 2.

Solution:
The claim is correct!
Before proving the claim, let's prove the following lemma:
Lemma: Every tree with $n \geq 3$ vertices has at least one vertex that it's degree is greater than 1 and at least one vertex that it's degree is exactly 1.
Proof:
(1) Every tree has at least one vertex that it's degree is exactly 1:
A tree is a connected acyclic graph, thus we know that the tree has exactly $n - 1$ edges, we also know that the sum of the degrees in the graph is $2m$ where m is the number of edges in the graph ⇒ the sum of the degrees is $2n - 2$.
Assume by contradiction that every vertex in tree has a degree of at least 2, then the sum of the degrees is at least $2n$, which contradicts the fact that the sum of the degrees is $2n - 2$, $\Rightarrow$ *In every tree, at least one vertex has a degree of* 1.
(2) Every tree has at least one vertex that it's degree is greater than 1:
Assume by contradiction that there exists a tree with n vertices, where every vertex h as a degree of 1, thus the sum of the edges in the graph is exactly $2n$, but we know for a fact that in every tree (acyclic and connected graph), the sum of the degrees is $2n - 2$. (→←). ⇒
Every tree has at least one vertex that it's degree is greater than 1.
Let $T = (V, E)$ be any tree where $|T| \geq 3$.
Assume by contradiction that T has $n \geq 3$ centers, start by removing all the vertices

with a degree equal to 1, in other words lets construct the graph T' described in the question, we know that all the centers in T are part of T' and are even centers in T'.
T' is also a tree with n centers according to our assumption, a center's degree is greater than 1 for sure, because if it wasn't it would get removed when creating T'.
Treat T' as the new T and re do the process of removing the vertices with degree equal to 1, no centers are removed and all the centers are still centers in the new tree.
Repeat the process until the tree is formed of centers only.
Let $T_f = (V_f, E_f)$ be the final tree where V_f is the set of vertices of T_f and it includes only centers.
According to our assumption and because no centers are lost in the process of removing vertices from T to T_f, the size of the set V_f is exactly n, that is the total number of centers in T. According to our assumption, it is greater or equal to 3. Thus, T_f is a tree of at least 3 vertices where all the vertices are centers of the tree.
According to the lemma proven above, every tree with at least 3 vertices has at least one vertex of degree 1 and at least one vertex of degree greater than 1, thus $\exists v \in V_f$ where v is a center and it's degree is exactly 1. This is not possible, because we know for a fact that every center has a degree of at least 2. ($\rightarrow\leftarrow$)
$\Rightarrow$The total number of centers in a tree can't be greater than 2. ■

Question 5:

The question involves scanning rooted binary trees. We saw in the tutorial three accepted ways to print the keys of a rooted tree: $Preorder$, $Postorder$ and for the special case of binary tree $Inorder$. You are asked to program a constructor for the class $Tree$ represented using $2-rooted-tree$. The input for the constructor are two arrays, where the first array P includes the keys to the new tree being built ordered $Preorder$ and the second array I includes the same keys ordered $Inorder$.

A tree that is built using this constructor will be a tree that if we run the relevant scanning algorithms from the tutorial, we will get the same input of the algorithm printed on the screen. You are required to provide pseudocode, an explanation of how the algorithm works and analyze the time complexity of the algorithm.

Solution:

We will suggest a constructor that builds a tree based on the $Preorder$ and $Inorder$ scan.

The idea is to build the tree recursively, and treating the left child/right child of any vertex as a sub-tree in itself.

Before starting with the solution, we should note the following:

1) The tree built is a data structure that is created using a single node that is the root of the tree.

2) The nodes are used to store some satellite data inside of them, we assume that this data is the data stored in the input arrays.

3) We assume that there exists a constructor for the nodes that creates a node based on the value and the left and right children.

4) The nodes are connected with each other using left and right attributes, in other words, each node has left and right attributes connecting the nodes together forming a graph that is acyclic and connected ⇒ a tree.

Here is the pseudocode:

```
tree(in_arr, pre_arr):
n = in_arr.length
pre_idx = [0]
root = build_recursively(in_arr, pre_arr, pre_idx, 0, n-1)
return root

build_recursively(in_arr, pre_arr, pre_idx, left, right):
if left > right do:
  return NIL
root = new Node(pre_arr[pre_idx[0]])
index = find(in_arr, pre_arr[pre_idx[0]], left, right)
pre_idx[0] += 1
root.left = build_recursively(in_arr, pre_arr, pre_idx,
                                  left, index - 1)
root.right = build_recursively(in_arr, pre_arr, pre_idx,
                                  index + 1, right)
return root

find(arr, val, left, right):
for i = left to right do:
  if arr[i] == val do:
    return i
return -1
```

Explanation of the pseudocode:
The idea behind the code provided above is to split the problem into three parts, the first part is locating the root, the second is building the left sub-tree and the third is building the right sub-tree. That's why we decided on the recursive approach so it is easier to handle the sub-trees problem.
What we did was we locate the root by approaching the first element in the $Preorder$ arr, we then located the root in the $Inorder$ arr and from that we can build the left sub-tree because it is built from all the elements to the left of the root in $Inorder$.
In a similar way we build the right sub-right tree, we then set the left and the right children of the root to be the appropriate roots of the recursive calls.
We choose pre_idx to be an array instead of an integer in order to have it shared through the recursive calls, this can be done in other ways too.

Time-complexity:
We can easily see that the constructor of building the tree takes $O(n^2)$, this is because in line 1,2,4 it does $O(1)$ operations while in line 3 a call to a recursive function is called, that function is $build_recursivly$.
$build_recursivly$ first finds the index of the relevant vertex in the array in_arr, this search is done in $O(n)$, since in the worst case it will iterate over all the array in order to find the relevant index, then it builds the left and right sub-trees recursively, so in total each vertex is the "$main\ vertex$" of the call exactly once, and each time $O(n)$ operations are done (searching in the array), thus in total we have $O(n^2)$.

A better and a more efficient approach to the problem is to use the same logic shown above, but using a hash-map to map the values in the in_arr instead of using an array, this way we can waiver the use of the method $find(arr, var, left, right)$ and get a total of $O(n)$ instead of $O(n^2)$.

Notice that our approach uses slicing on array and we assume that the slicing operation can be done in $O(1)$, this is true for some programming languages but not for all, thus the work around is to use indices instead of slicing in order to get the relevant bounds.